Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Ingeniería

Carrera: Ingeniería de Software

Asignatura: Aplicaciones Distribuidas

Código de la actividad: TA-IND-01

Informe Académico Individual:
Análisis crítico de CockroachDB frente a Apache Cassandra

Estudiante: Toaquiza Zambrano Belinda Betzabeth

Paralelo: A

Fecha: 13 de noviembre de 2026

Resumen

[PENDIENTE: Escribe entre 150 y 200 palabras: sistema analizado (CockroachDB), enfoque (comparación con Apache Cassandra, conceptos de la Unidad 1, experimento propio) y conclusiones principales. Redáctalo al final, cuando ya tengas los resultados.]

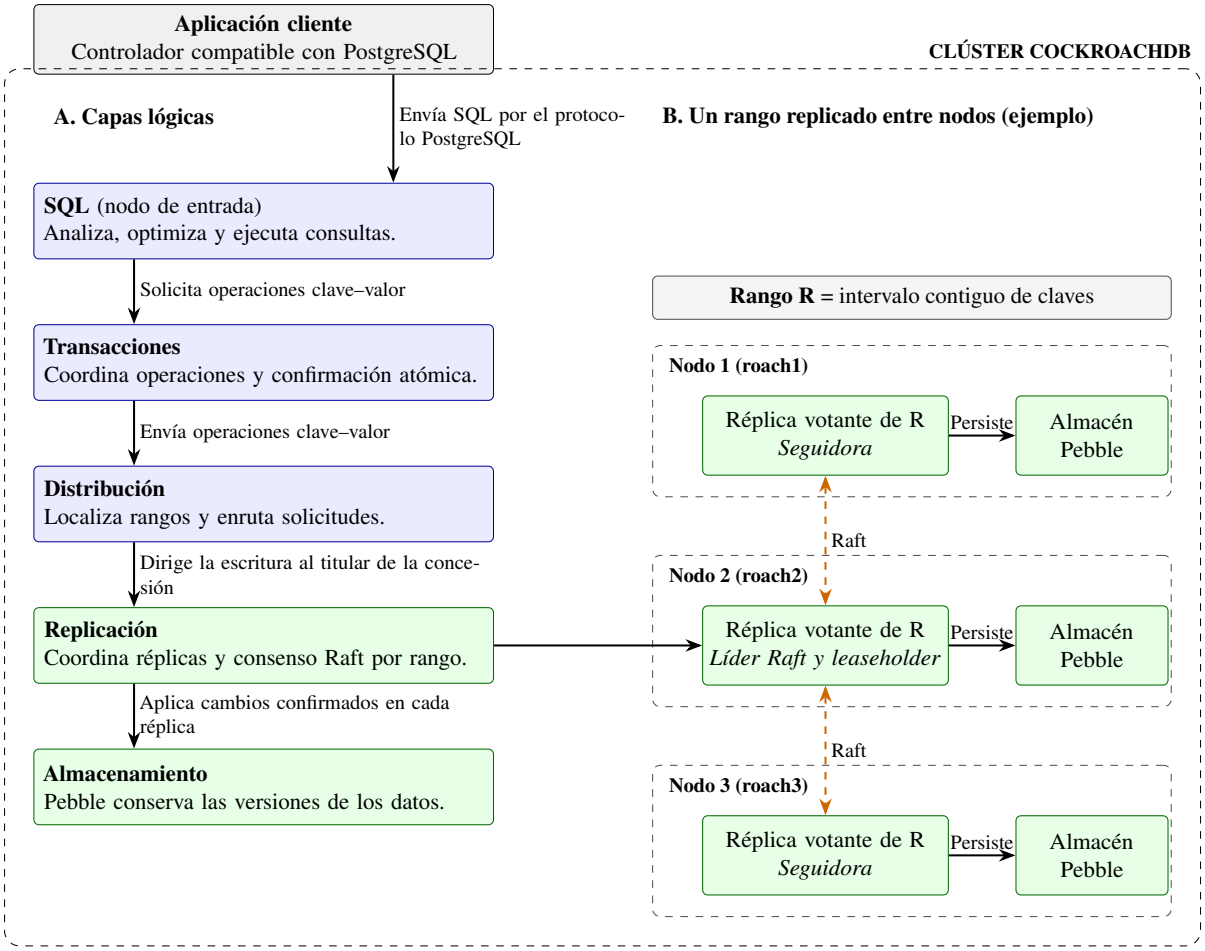
Palabras clave: [PENDIENTE: 4 a 6 palabras clave].

1. Introducción

[PENDIENTE: Problema que resuelve CockroachDB, quién lo usa, escala de operación, motivo de la elección y organización del informe.]

2. Arquitectura del sistema

La aplicación cliente envía consultas a un nodo de CockroachDB mediante un controlador compatible con PostgreSQL. La capa SQL analiza y ejecuta las consultas, mientras que la capa de transacciones coordina las operaciones de lectura y escritura. La capa de distribución localiza los rangos de claves y dirige cada operación a las réplicas correspondientes, como se describe en la arquitectura del sistema [1]. En el ejemplo del diagrama, la réplica que coordina la escritura es el líder Raft y se comunica con las otras dos réplicas. La escritura se confirma al alcanzar el quórum requerido, es decir, al replicarse en al menos dos de los tres nodos [2]. Finalmente, cada nodo conserva los datos en su almacenamiento local Pebble [3].



Leyenda: flecha continua = solicitud o aplicación local; flecha discontinua naranja = comunicación Raft; cajas azules = procesamiento y enrutamiento; cajas verdes = replicación y almacenamiento. Los roles de Raft se asignan por rango; el líder mostrado es un ejemplo.

Figura 1: Arquitectura lógica de CockroachDB y réplica de un rango en el clúster de tres nodos del experimento (elaboración propia).

El esquema preliminar mostraba las cinco capas de CockroachDB y un ejemplo de un rango replicado en tres nodos. La versión final amplió esa vista al detallar la función de cada capa y el recorrido de una solicitud desde la aplicación cliente hasta el almacenamiento. También distinguió la réplica líder de las seguidoras y mostró cómo se comunican mediante Raft, cuántas réplicas se necesitan para confirmar una escritura y cómo se guardan los cambios en Pebble en cada nodo.

3. Análisis con los conceptos de la Unidad 1

3.1. Transparencias de distribución

La transparencia de distribución describe cuánto puede usar una aplicación los datos sin tener que conocer su ubicación física, el número de réplicas o los cambios internos del clúster. En ambos motores hay transparencias parciales: la interfaz simplifica el acceso, pero las decisiones sobre consistencia, disponibilidad y recuperación siguen siendo visibles para quien opera o programa el sistema.

Acceso. CockroachDB ofrece una interfaz SQL compatible con el protocolo de red de PostgreSQL; traduce las consultas en solicitudes sobre su almacenamiento distribuido. Cassandra expone CQL mediante

su protocolo binario nativo. En los dos casos, el usuario trabaja con una interfaz lógica sin enviar manualmente cada consulta a todas las réplicas [1], [4], [5].

Ubicación. Ambos sistemas ocultan en gran medida dónde reside cada dato. CockroachDB divide el espacio de claves en rangos y distribuye el encaminamiento entre los nodos. Cassandra calcula el token de la clave de partición y entrega la solicitud a las réplicas pertinentes; cualquier nodo puede actuar como coordinador. La ubicación física no forma parte de la consulta normal, aunque sí puede ser relevante para la configuración y la observabilidad [1], [6].

Migración. La ubicación lógica también permanece separada de la consulta cuando cambian las réplicas. CockroachDB puede reequilibrar rangos entre nodos; Cassandra distribuye tokens en el anillo y permite añadir, mover o retirar nodos. Esta transparencia es operativa, no absoluta: el cambio de topología requiere configuración y puede involucrar movimiento de datos, carga de red y tareas de mantenimiento [2], [6].

Reubicación. La ocultación de una transferencia mientras hay carga es parcial. CockroachDB describe la transferencia de instantáneas y la puesta al día de réplicas durante el rebalanceo. En Cassandra, los cambios del anillo y la reparación pueden transferir datos entre nodos. Las fuentes describen estos mecanismos, pero no garantizan que toda consulta en curso termine sin demora o reintento; esa continuidad tampoco se midió en esta práctica [2], [6], [7].

Replicación. CockroachDB administra réplicas de rangos mediante Raft y el cliente no selecciona los nodos que deben confirmar una escritura. Cassandra replica cada partición según la estrategia del espacio de claves, pero expone al cliente los niveles de consistencia (`ONE`, `QUORUM`, `ALL`, entre otros), que fijan cuántas respuestas espera el coordinador. Por eso Cassandra automatiza la copia de datos, aunque deja visible una parte mayor de la política de lectura y escritura [2], [6].

Concurrencia. CockroachDB aplica aislamiento `SERIALIZABLE` por defecto: el efecto de transacciones concurrentes debe corresponder a algún orden serial. La contención puede provocar reintentos y, si no se resuelve automáticamente, un error visible para la aplicación. Cassandra resuelve mutaciones ordinarias concurrentes por marcas de tiempo y regla de último escrito; no ofrece por ello una transacción serializable global entre particiones. Para operaciones condicionales de una partición ofrece transacciones ligeras basadas en Paxos. Así, en los dos motores la aplicación puede tener que manejar contención o elegir una garantía concreta [4], [6], [8].

Fallos. La abstracción también es parcial. CockroachDB puede seguir atendiendo operaciones que conservan mayoría de votantes para el rango, pero puede rechazar una lectura actual o una escritura si falta quórum; también puede perderse la conexión al nodo usado por el cliente. Cassandra deja que el coordinador atienda según el nivel elegido, por lo que una operación con `ONE` puede responder cuando niveles mayores no alcanzan sus réplicas requeridas. Los errores de disponibilidad, los tiempos de espera y los reintentos no quedan ocultos por completo [2], [6].

3.2. CAP y PACELC

CAP y PACELC describen compromisos bajo supuestos concretos, no una etiqueta independiente de la operación. En lo que sigue, la clasificación se limita a lecturas actuales y escrituras ordinarias sobre datos con tres réplicas; las conclusiones empíricas se restringen a las operaciones registradas en los experimentos [2], [6], [8].

Durante una partición de red, CockroachDB prioriza mantener la consistencia de las operaciones que confirma: Raft exige mayoría de las réplicas del rango. En una disposición de tres réplicas, el lado con dos puede avanzar y el lado con una no puede confirmar una escritura que requiere quórum. Por ello, para estas operaciones CockroachDB presenta el perfil CP: tolera la partición, pero sacrifica disponibilidad en la minoría. En Cassandra el compromiso depende del nivel por operación. La documentación describe su orientación AP; en la práctica, `ONE` puede responder en cada lado si cada uno alcanza una réplica, mientras que `QUORUM` solo alcanza el mínimo en la mayoría y `ALL` requiere las tres. Con niveles más exigentes, Cassandra también sacrifica disponibilidad en el lado que no reúne las respuestas [2], [6], [8].

Los ensayos concuerdan con ese contraste, dentro del alcance de las salidas observadas. Las salidas propias constan en `experimento/comandos.md`, apartados de E1 y E2. En E1, CockroachDB leyó y escribió con un nodo detenido; al detener dos, la consulta devolvió `replica unavailable`. En E2, el lado mayoritario confirmó la fila 31 y la lectura actual desde `roach3` aislado falló con `lost quorum`; el `INSERT` de la fila 32 quedó sin respuesta visible, así que no se afirma si se confirmó. En Cassandra, la partición 2+1 mostró que `ONE` completó lecturas y escrituras en ambos lados; `QUORUM` solo en el lado con dos réplicas; y `ALL` falló en ese lado y no se probó en el aislado. Tras reconectar, `ALL` devolvió las filas 21, 23 y 24. Como se usaron claves distintas, la prueba no demuestra cómo se resolverían escrituras concurrentes sobre la misma clave [2], [6].

PACELC añade la pregunta del comportamiento sin partición: consistencia o menor latencia. Para operaciones actuales, CockroachDB mantiene coordinación de consenso y confirmación por mayoría, lo que puede añadir viajes de red; ofrece lecturas históricas de seguidores cuando se acepta menor frescura. El trabajo de VanBenschoten et al. estudia configuraciones de localidad y lecturas locales consistentes en despliegues multirregión; sus resultados explican una optimización posible, pero no predicen la latencia de este clúster local. La inferencia para las operaciones actuales de esta práctica es un perfil PC/EC: ante partición conserva consistencia en la mayoría y, aun sin partición, conserva consistencia mediante coordinación [2], [9], [10].

Cassandra permite escoger cuánto espera el coordinador. `ONE` exige menos respuestas y puede favorecer menor espera; `QUORUM` y `ALL` exigen más confirmaciones y restringen las operaciones que pueden responder. Por ello, su perfil sin partición no es único: el nivel elegido mueve el compromiso hacia latencia o consistencia. Ferreira et al. compararon niveles de consistencia en Cassandra y otros motores en despliegues de nube, y hallaron diferencias de rendimiento asociadas a la configuración; da Silva y Lima evaluaron sistemas en un clúster de placas de bajo consumo. Nejati SharifAldin y Nayebi Pour compararon los niveles nativos con un middleware causal externo. Estos estudios usan cargas y topologías distintas y respaldan el compromiso cualitativo, no una predicción numérica para Docker. En la práctica no se midió latencia; por tanto, la clasificación PL/EL para `ONE` y PC/EC para niveles de quórum se presenta como inferencia documental y no como resultado de rendimiento del experimento [6], [11], [12], [13].

3.3. Modelos de comunicación

CockroachDB combina el modelo cliente-servidor hacia la aplicación con coordinación distribuida entre nodos. El cliente envía SQL por el protocolo PostgreSQL a un nodo; internamente, los nodos participantes coordinan transacciones y replicación por rangos mediante Raft. La llamada del cliente es síncrona en el sentido de que espera éxito o error de la operación; la escritura no se confirma antes de obtener el quórum requerido. La replicación, el rebalanceo y la actualización de réplicas ocurren mediante intercambios de red entre nodos. El artículo de Kettaneh et al. describe el mecanismo Leader Leases y su detector distribuido de fallos, mientras el documento de multirregión analiza el efecto de la localidad en las transacciones. Esta descripción es arquitectónica; las capturas de la práctica no exponen los mensajes internos ni su temporización [1], [2], [4], [10], [14].

Cassandra también recibe solicitudes de clientes en un modelo cliente-servidor, pero su clúster opera

entre pares: un nodo accesible puede coordinar cada petición y distribuirla a las réplicas. El protocolo nativo transporta mensajes CQL desde el cliente; entre nodos se usa mensajería internodo, y gossip intercambia información de membresía y estado. La respuesta al cliente es síncrona respecto al nivel CQL seleccionado: el coordinador espera el número requerido de respuestas. En paralelo, gossip, hinted handoff y reparación participan en la propagación o convergencia posterior, por lo que el sistema combina comunicación de petición-respuesta con trabajo de fondo asíncrono [5], [6], [7], [15], [16].

3.4. Tolerancia a fallos

Con tres votantes para un rango, CockroachDB puede tolerar la pérdida de uno mientras los otros dos se comunican. Si se pierde la mayoría, no confirma nuevas escrituras de ese rango; otras partes del clúster pueden seguir funcionando si conservan sus propios quórums. Raft mantiene el registro replicado y el sistema puede transferir liderazgo y concesiones después de detectar que un nodo no está disponible. El artículo de Kettaneh et al. describe Leader Leases con una detección de vitalidad entre nodos. Como la práctica no inspeccionó métricas internas ni el algoritmo exacto de detección activo, los resultados externos no permiten verificar ese mecanismo directamente [2], [14].

En E1, con `roach2` detenido, CockroachDB mantuvo lectura y escritura; con `roach2` y `roach3` detenidos, la consulta no alcanzó quórum. Al iniciar los contenedores, el servicio SQL volvió a responder y las filas confirmadas siguieron disponibles. Sin embargo, `node status` mostró un estado anómalo de `roach1` hasta que se reinició; la causa de ese estado no quedó establecida. En E2, con `roach3` desconectado de la red pero aún encendido, la escritura desde la mayoría funcionó y la lectura actual desde el nodo aislado devolvió `lost quorum`. Ese escenario comprueba una partición de comunicación, distinta de detener el proceso [2].

Cassandra detecta disponibilidad de pares a partir de heartbeats y gossip. Si una réplica no está disponible, el coordinador puede conservar una mutación pendiente mediante hinted handoff; ese mecanismo es de mejor esfuerzo. La reparación anti-entropía compara rangos con árboles de Merkle y transmite diferencias, y requiere ejecución operativa con `nodetool repair`. En E1, con RF=3, `ONE` y `QUORUM` funcionaron tras detener un nodo, mientras `ALL` falló; con dos nodos detenidos, solo `ONE` completó la lectura y la escritura comprobada. Tras iniciar nodos, el estado llegó a tres `UN` y `ALL` leyó las filas confirmadas. En E2, los tres nodos volvieron a `UN`, pero las consultas posteriores no verifican por sí mismas si se usaron hints o repair; el mecanismo exacto de convergencia quedó sin medir [6], [7], [8].

Como caso operativo real, Discord describió que su clúster Cassandra de mensajes llegó a 177 nodos y trillones de mensajes, con latencia impredecible, alta carga de mantenimiento y degradación en cascada asociada a particiones calientes y consultas con consistencia de quórum. La publicación relata una degradación de rendimiento y operación, no una caída total del servicio ni una falla de consenso. El caso muestra que replicación y disponibilidad no evitan problemas producidos por patrones de acceso sesgados y carga; no permite trasladar sus magnitudes a esta práctica [17]. El análisis de latencia permanece cualitativo porque en los experimentos no se registraron mediciones temporales comparables.

4. Experimentación propia

4.1. Entorno y justificación

[PENDIENTE: Cómo levantaste el sistema (por ejemplo, Docker), versión y por qué elegiste ese entorno.]

4.2. Procedimiento

[PENDIENTE: Describe el experimento y el concepto de la Unidad 1 que ilustra.]

Listing 1: Comandos usados en el experimento (ejemplo de formato).

```
# Reemplaza este bloque por tus comandos reales.
# El archivo completo debe estar en experimento/ del repositorio.
```

4.3. Resultados

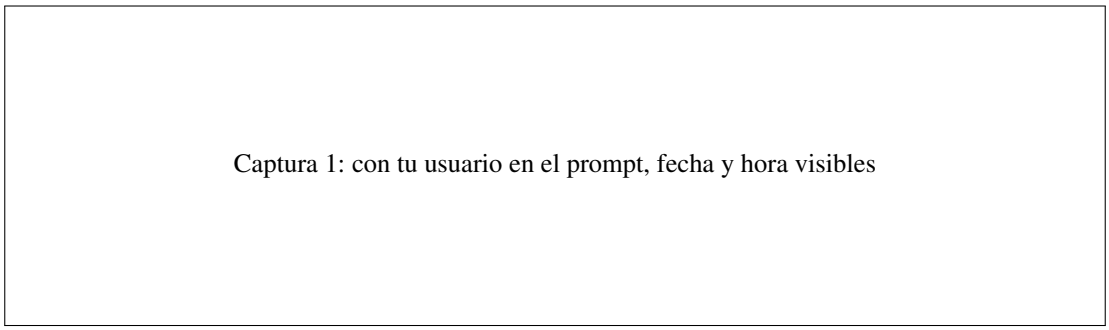


Figura 2: [PENDIENTE: Qué muestra la captura 1.]

[PENDIENTE: Incluye al menos 4 capturas propias de la ejecución real del sistema (duplica el bloque de figura anterior).]

4.4. Interpretación

[PENDIENTE: Interpreta los resultados a la luz de las transparencias, CAP/PACELC, comunicación y tolerancia a fallos de la sección anterior.]

5. Ejemplos del entorno del estudiante

5.1. Ejemplo 1: [PENDIENTE: sistema]

[PENDIENTE: Qué observaste (caída, mensaje tardío, saldo que tardó en actualizarse) y con qué concepto se relaciona.]

5.2. Ejemplo 2: [PENDIENTE: sistema]

[PENDIENTE: Observación y concepto asociado.]

6. Análisis comparativo

Tabla 1: Comparación entre CockroachDB y Apache Cassandra.

Criterio	CockroachDB	Apache Cassandra
Transparencias de distribución	Acceso SQL y ubicación lógica de rangos quedan mayormente ocultos; la pérdida de quórum y los reintentos sí llegan a la aplicación [1], [2], [4].	CQL y tokens ocultan la ubicación, pero el cliente configura el nivel de consistencia y puede recibir errores de disponibilidad [5], [6].
CAP y PACELC	En E2, la mayoría confirmó una escritura; la réplica aislada no pudo completar la lectura actual. Perfil CP y PC/EC para las operaciones actuales, como inferencia del quórum Raft [2].	E2: ONE operó en ambos lados, QUORUM solo con dos réplicas y ALL falló en la mayoría. La elección del nivel cambia disponibilidad y coordinación; no se midió latencia [6].
Modelo de comunicación	Cliente-servidor con SQL/PostgreSQL; los nodos coordinan rangos mediante Raft y esperan confirmación de quórum para escrituras [2], [4].	Cliente-servidor con CQL hacia un coordinador y comunicación entre pares para réplicas; la respuesta espera el nivel elegido y gossip opera en segundo plano [5], [6], [16].
Replicación y consenso	Replicación por rangos con consenso Raft; se requieren dos de tres votantes para conservar el quórum del rango [2].	Replicación por estrategia de clave de espacio y factor RF=3; los niveles ONE, QUORUM y ALL determinan respuestas requeridas, sin consenso Raft global [6].
Tolerancia a fallos	E1: con un nodo detenido hubo lecturas y escrituras; con dos, se perdió quórum. E2: la mayoría escribió y la lectura actual aislada falló [2].	E1 y E2 mostraron disponibilidad dependiente del nivel y de las réplicas alcanzables. Tras reconectar, se verificaron tres nodos UN; el mecanismo exacto de convergencia no se midió [6], [7].

La diferencia práctica principal es dónde se fija el compromiso de disponibilidad. CockroachDB conserva una regla de quórum Raft para las operaciones actuales; Cassandra permite seleccionar el número de réplicas que deben responder por operación. Los ensayos concuerdan con esa diferencia en las condiciones probadas: CockroachDB dejó de confirmar operaciones al perder mayoría, mientras Cassandra permitió operaciones `ONE` en ambos lados de la partición. Los resultados no miden rendimiento ni prueban escrituras concurrentes sobre la misma clave [2], [6].

7. Postura crítica

7.1. Fortalezas

[PENDIENTE:]

7.2. Limitaciones y compromisos (*trade-offs*)

[PENDIENTE:]

7.3. Contextos en los que no sería la mejor opción

[PENDIENTE:]

8. Conclusiones

[PENDIENTE: Conclusiones concretas basadas en el análisis y lo observado en el experimento.]

Declaración de uso de inteligencia artificial

[PENDIENTE: Herramienta y versión, para qué la usaste y cómo verificaste lo obtenido.]

Referencias

[1] Cockroach Labs. «Architecture Overview,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/architecture/overview

[2] Cockroach Labs. «Replication Layer,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/architecture/replication-layer

[3] Cockroach Labs. «Storage Layer,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/architecture/storage-layer

[4] Cockroach Labs. «Developer Basics,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/developer-basics

[5] Apache Software Foundation. «Native Protocol Versions,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/reference/native-protocol.html

[6] Apache Software Foundation. «Dynamo,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html

[7] Apache Software Foundation. «Repair,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/managing/operating/repair.html

[8] Apache Software Foundation. «Guarantees,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/guarantees.html

[9] Cockroach Labs. «Follower Reads,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/follower-reads

[10] N. VanBenschoten et al., «Enabling the Next Generation of Multi-Region Applications with CockroachDB,» en *Proceedings of the 2022 International Conference on Management of Data*, Association for Computing Machinery, 2022. DOI: 10.1145/3514221.3526053 dirección: https://dl.acm.org/doi/10.1145/3514221.3526053

[11] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, 2024, **artno** 158. DOI: 10.1186/s13677-024-00716-7 dirección: https://link.springer.com/article/10.1186/s13677-024-00716-7

[12] L. F. da Silva y J. V. F. Lima, «An evaluation of relational and NoSQL distributed databases on a low-power cluster,» *The Journal of Supercomputing*, vol. 79, n.º 12, págs. 13 402-13 420, 2023. DOI: 10.1007/s11227-023-05166-7 dirección: https://link.springer.com/article/10.1007/s11227-023-05166-7

[13] H. Nejati SharifAldin y F. Nayebi Pour, «Middleware-enforced Timed Causal Consistency for Apache Cassandra: An energy–performance–consistency evaluation against static consistency levels using YCSB,» *Software Impacts*, vol. 28, 2026, **artno** 100817. DOI: 10.1016/j.simpa.2026.100817 dirección: https://www.sciencedirect.com/science/article/pii/S2665963826000072

[14] I. Kettaneh, T. Radeva, A. Ajmani, S. Bhola, N. VanBenschoten y R. Taft, «Scalable Leader Leases For Multi Consensus Groups in CockroachDB,» en *Companion of the International Conference on Management of Data*, Association for Computing Machinery, 2026. DOI: 10.1145/3788853.3803081 dirección: https://dl.acm.org/doi/10.1145/3788853.3803081

[15] Apache Software Foundation. «Architecture,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/

[16] Apache Software Foundation. «Improved Internode Messaging,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/messaging.html

[17] B. Ingram. «How Discord Stores Trillions of Messages,» Discord Engineering, visitado 13 de nov. de 2026. dirección: https://discord.com/blog/how-discord-stores-trillions-of-messages

Anexo A. Esquema inicial a mano

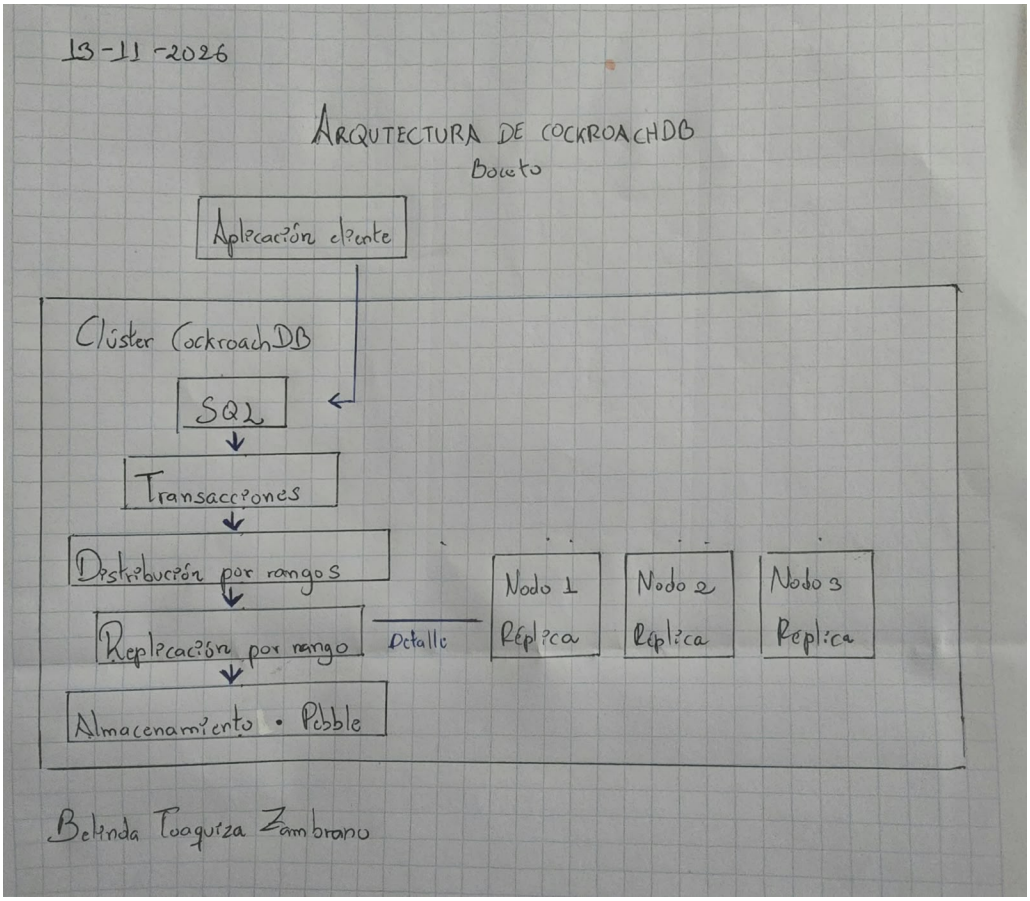
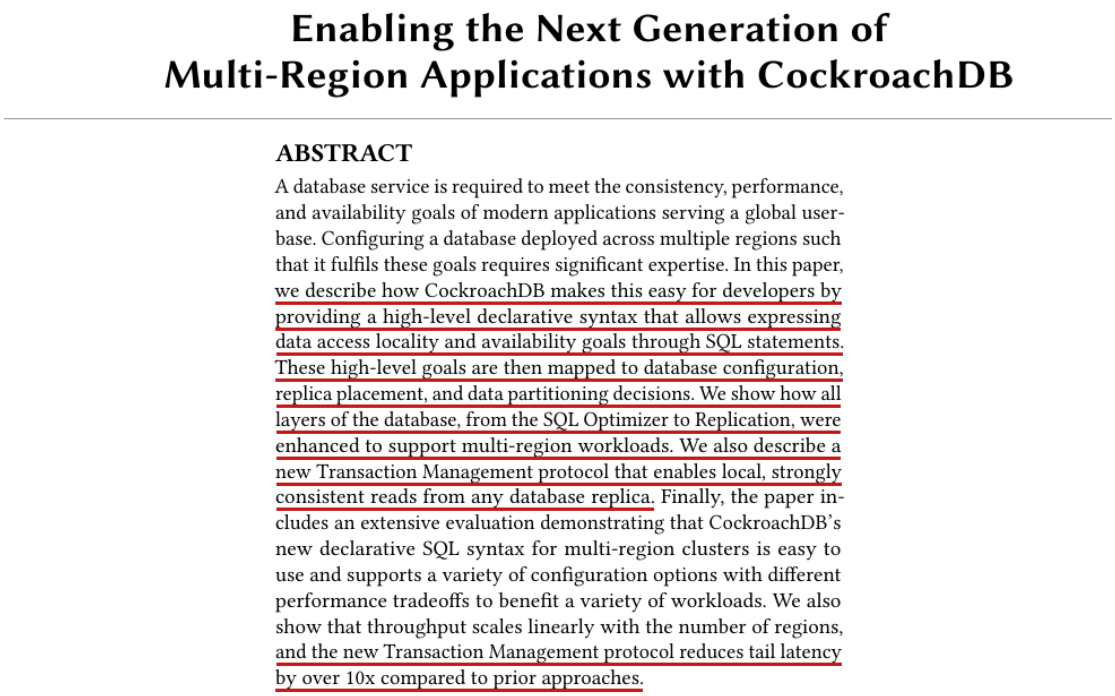


Figura 3: Esquema inicial de la arquitectura de CockroachDB, dibujado a mano.

Anexo B. Capturas de lectura de las fuentes

En cada captura se subrayó el pasaje empleado en la sección 3 y se indican el número de la referencia y la página.

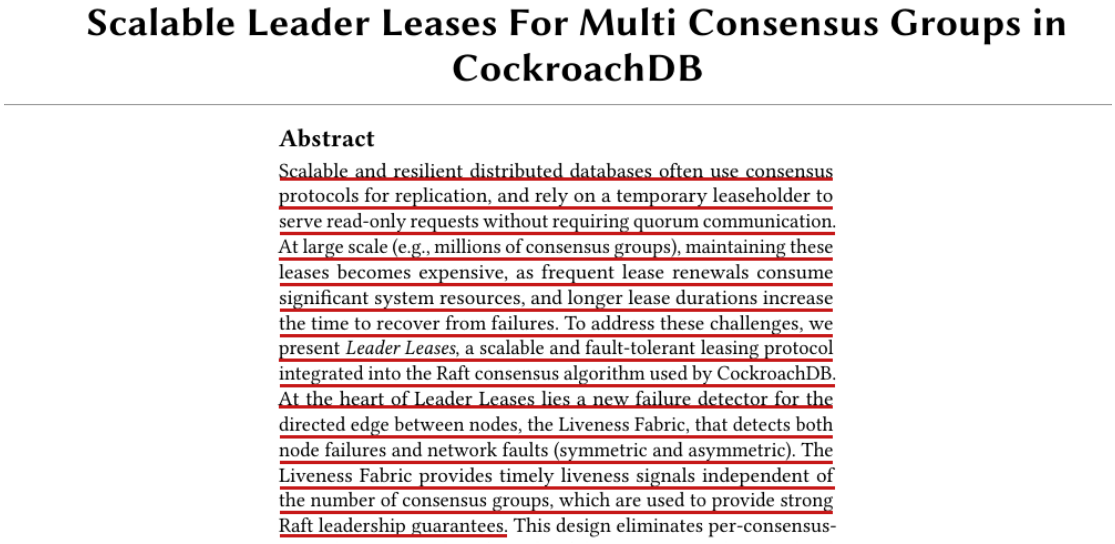


Enabling the Next Generation of
Multi-Region Applications with CockroachDB

ABSTRACT

A database service is required to meet the consistency, performance, and availability goals of modern applications serving a global user-base. Configuring a database deployed across multiple regions such that it fulfils these goals requires significant expertise. In this paper, we describe how CockroachDB makes this easy for developers by providing a high-level declarative syntax that allows expressing data access locality and availability goals through SQL statements. These high-level goals are then mapped to database configuration, replica placement, and data partitioning decisions. We show how all layers of the database, from the SQL Optimizer to Replication, were enhanced to support multi-region workloads. We also describe a new Transaction Management protocol that enables local, strongly consistent reads from any database replica. Finally, the paper includes an extensive evaluation demonstrating that CockroachDB's new declarative SQL syntax for multi-region clusters is easy to use and supports a variety of configuration options with different performance tradeoffs to benefit a variety of workloads. We also show that throughput scales linearly with the number of regions, and the new Transaction Management protocol reduces tail latency by over 10x compared to prior approaches.

Figura 4: Fuente citada [10], p. 1: se subrayó la descripción de la sintaxis declarativa que expresa objetivos de localidad y disponibilidad de los datos y la del protocolo de transacciones que habilita lecturas locales fuertemente consistentes desde cualquier réplica; ambos pasajes sostienen la afirmación de la sección 3.2 sobre configuraciones de localidad y lecturas locales en despliegues multirregión.



Scalable Leader Leases For Multi Consensus Groups in
CockroachDB

Abstract

Scalable and resilient distributed databases often use consensus protocols for replication, and rely on a temporary leaseholder to serve read-only requests without requiring quorum communication. At large scale (e.g., millions of consensus groups), maintaining these leases becomes expensive, as frequent lease renewals consume significant system resources, and longer lease durations increase the time to recover from failures. To address these challenges, we present *Leader Leases*, a scalable and fault-tolerant leasing protocol integrated into the Raft consensus algorithm used by CockroachDB. At the heart of Leader Leases lies a new failure detector for the directed edge between nodes, the Liveness Fabric, that detects both node failures and network faults (symmetric and asymmetric). The Liveness Fabric provides timely liveness signals independent of the number of consensus groups, which are used to provide strong Raft leadership guarantees. This design eliminates per-consensus-

Figura 5: Fuente citada [14], p. 1: se subrayó la presentación del mecanismo Leader Leases, su integración en el algoritmo de consenso Raft y el detector distribuido de fallos Liveness Fabric; respalda lo expuesto en las secciones 3.3 y 3.4 sobre el arrendamiento del líder y su detección de vitalidad entre nodos.

2 concurrent users. Comparatively, in its worst scenarios, Cassandra shows a 36% decrease (from around 1500 to around 970) with 2 concurrent users, and Redis demonstrates a 25% decrease (from 1389 to 1083) with 2 concurrent users.

Throughput in reading operations. Cassandra's performance is significantly lower compared to writing operations. It experiences around 65% of decrease (from 109 to 37) in the number of requests per second processed by the system in its worst-case scenario with 6 concurrent users. Meanwhile, MongoDB exhibits a 43% decrease (from 160 to 90) in its worst-case scenario with 2 users. The divergence of Cassandra's performance can be attributed to its in-memory storage, which requires runtime write validation across replicas, causing slowdowns in reads under strong consistency. In addition,

Figura 6: Fuente citada [11], p. 8: se subrayó el resultado que atribuye a Cassandra una caída del 36 % en el rendimiento de escritura y la explicación de la validación de escritura entre réplicas como causa de las lecturas lentas con consistencia fuerte; sustenta la afirmación de la sección 3.2 sobre diferencias de rendimiento asociadas a la configuración de consistencia.

The Journal of Supercomputing (2023) 79:13402–13420
https://doi.org/10.1007/s11227-023-05166-7

Check for updates

An evaluation of relational and NoSQL distributed databases on a low-power cluster

Lucas Ferreira da Silva[1] · **João V. F. Lima**[1]

Accepted: 4 March 2023 / Published online: 23 March 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

The constant growth of social media, unconventional web technologies, mobile applications, and Internet of Things (IoT) devices create challenges for cloud data systems in order to support huge datasets and very high request rates. NoSQL databases, such as Cassandra and HBase, and relational SQL databases with replication, such as Citus/PostgreSQL, have been used to increase horizontal scalability and high availability of data store systems. In this paper, we evaluated three distributed databases on a low-power low-cost cluster of commodity Single-Board Computers (SBC): relational Citus/PostgreSQL and NoSQL databases Cassandra and HBase. The cluster has 15 Raspberry Pi 3 nodes with Docker Swarm orchestration tool for service deployment and ingress load balancing over SBCs. We believe that a low-cost SBC cluster can support cloud serving goals such as scale-out, elasticity, and high availability. Experimental results clearly demonstrated that there is a trade-off between performance and replication, which provides availability and partition tolerance. Besides, both properties are essential in the context of distributed systems with low-power boards. Cassandra attained better results with its consistency levels specified by the client. Both Citus and HBase enable consistency but it penalizes performance as the number of replicas increases.

Figura 7: Fuente citada [12], p. 1: se subrayó la evaluación de tres bases de datos distribuidas en un clúster de placas de bajo consumo con Docker Swarm y el resultado de Cassandra con los niveles de consistencia fijados por el cliente; respalda la mención de la sección 3.2 al clúster de placas de bajo consumo.

Software Impacts 28 (2026) 100817



Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts



Software update

Middleware-enforced Timed Causal Consistency for Apache Cassandra: An energy–performance–consistency evaluation against static consistency levels using YCSB



Hesam Nejati SharifAldin *, Farnoush Nayebi Pour

Department of Computer Engineering, Mashhad Branch, Islamic Azad University, Mashhad, Iran

ARTICLE INFO

Keywords:
Apache Cassandra
Consistency
Replication
Frequency-scaling
Energy–performance
Tools and monitoring

ABSTRACT

We present a reproducible and open-source middleware-enforced consistency model — Strict Timed Causal Consistency (STCC) — designed as an external consistency layer for Cassandra, systematically compared against built-in static levels (ALL, QUORUM, ONE) in terms of energy, performance, and consistency trade-offs under YCSB workloads. Cassandra's default consistency levels (e.g., ALL, QUORUM, ONE) are statically configured and unable to adapt to runtime variations in workload intensity or network conditions. To address this limitation, we introduce a fully functional middleware implementing *Strict Timed Causal Consistency* (STCC) — a hybrid model that combines client- and server-side causal guarantees with a configurable freshness bound Δ. STCC is non-intrusive and integrates seamlessly with unmodified Cassandra deployments, enabling adaptive consistency at runtime. The middleware integrates seamlessly with unmodified Cassandra instances, offering tunable freshness bounds (Δ) and causal guarantees across geo-replicated nodes without modifying the core database engine. The package includes automation tools for YCSB-based workload execution, CPU frequency control via `cpupower`, and energy monitoring using SNMP-enabled PDUs and `dstat`. This framework targets cloud engineers, distributed systems researchers, and middleware designers seeking to analyze or optimize consistency–latency–energy trade-offs under realistic deployment conditions. Our experiments validated on a 24-node Cassandra cluster distributed across three Cassandra's data centers, the system is publicly released with full documentation to support reproducible and extensible experimentation.

Figura 8: Fuente citada [13], p. 1: se subrayó la presentación de STCC como capa de consistencia externa comparada con los niveles nativos de Cassandra y la descripción de esos niveles como estáticos; sustenta la afirmación de la sección 3.2 sobre la comparación con un middleware causal externo.